

Правила разработки контрола на TypeScript

Импорты

```
1 // Хорошо
2 import {Control, IControlOptions} from 'UI/Base'; // пример с контролем
3 import * as template from 'wml!MyModule/MyControl'; // пример с
  шаблоном

1 // Плохо
2 import Control = require('UI/Base:Controls');
3 import template = require('wml!MyModule/MyControl');
```

Интерфейсы

```
1 export interface ICheckableOptions {
2     value?: boolean;
3 }
4
5 /**
6  * Interface for 2-position control.
7  *
8  * @interface Controls/_toggle/interface/ICheckable
9  * @public
10 * @author Красильников А.С.
11 */
12 export interface ICheckable {
13     readonly '[Controls/_toggle/interface/ICheckable]': boolean;
14 }
15 /**
16  * @name Controls/_toggle/interface/ICheckable#value
17  * @cfg {Boolean} Current state.
18  */
19 /**
20  * @event Controls/_toggle/interface/ICheckable#valueChanged Occurs
  when value changes.
21  * @cfg {Boolean} New value.
22  */
```

1. Файл с интерфейсом экспортирует отдельный интерфейс с опциями. Название – имя интерфейса + Options.

```
1 export interface ICheckableOptions {
2     value?: boolean;
3 }
```

2. Файл с интерфейсом экспортирует сам интерфейс:

```
1 export interface ICheckable {...}
```

3. Всю документацию по интерфейсу опишите в файле интерфейса.
 4. Если интерфейс используется не более чем в одном классе, для него можно не создавать отдельный файл.
 5. Если интерфейс используется в рамках одной библиотеки, его размещают в директории interface внутри приватной папки библиотеки. И такой интерфейс должен экспортироваться библиотекой.
 6. Если интерфейс используется в нескольких библиотеках, его размещают в директории interface в корне интерфейсного модуля. И такой интерфейс должен экспортироваться библиотекой interface.
- Это правило используется в интерфейсном модуле [Controls](#). Папка interface необязательная. В корне интерфейсного модуля можно разместить общие для нескольких библиотек интерфейсы.

```
1 Controls/_toggle/interface/ICheckable.d.ts
```

7. В файле контрола, реализующего интерфейс – импортируйте интерфейс и пропишите в директиве implements.

```
1 import ICheckable from './interface/ICheckable';
```

```
1 class BigSeparator extends Control implements ICheckable {...}
```

8. Описание интерфейса в доклетах идет прямо перед его объявлением в коде, а описание опций, методов и событий — в конце файла.

```
1 /**
2  * Interface for 2-position control.
3  *
4  * @interface Controls/_toggle/interface/ICheckable
5  * @public
6  * @author Красильников А.С.
7  */
8 export interface ICheckable {
9     readonly '[Controls/_toggle/interface/ICheckable]': boolean;
10 }
11 /**
12  * @name Controls/_toggle/interface/ICheckable#value
13  * @cfg {Boolean} Current state.
14  */
15 /**
16  * @event Controls/_toggle/interface/ICheckable#valueChanged Occurs
17  * when value changes.
18  * @cfg {Boolean} New value.
19  */
```

9. Поле типа Boolean с именем интерфейса в квадратных скобках делается по двум причинам:

- В TS/ES6 нет нативной проверки на имплементацию экземпляром интерфейса (аналога instanceof для интерфейсов). Подобное поле с уникальным именем позволит писать проверки вида:

```
1 if (instance['[Controls/_toggle/interface/ICheckable]'])
```

- TS ругается на пустой интерфейс.

Интерфейсы для опций

1. Импортируйте опции интерфейса и всех контролов, от которых ведется наследование.

```
1 import {ICheckable, ICheckableOptions} from './interface/ICheckable';
```

2. Каждый контрол и интерфейс экспортирует свой набор опций. Для этого объявите интерфейс опций и соберите его из всех родительских интерфейсов-опций. Пример для наследника Control и интерфейса ICheckable:

```
1 export interface IBigSeparatorOptions extends IControlOptions,
  ICheckableOptions {...}
```

3. Поле _options описано в базовом UI/Base:Control, и его можно не описывать его в своих контролах.

4. Интерфейс опций используйте в [хуках жизненного цикла](#).

```
1 protected _beforeMount(newOptions: IBigSeparatorOptions): void {
2     this._iconChangedValue(newOptions.value);
3 }
4
5 protected _beforeUpdate(newOptions: IBigSeparatorOptions): void {
6     this._iconChangedValue(newOptions.value);
7 }
```

Классы и методы

1. Контрол — это класс. Базовый Control – Generic class, который принимает интерфейс опций.

```
1 // Хорошо
2 class BigSeparator extends Control<IBigSeparatorOptions> implements
  ICheckable {...}
1 // Плохо
2 var BigSeparator = Control.extend({...})
```

2. Приватные методы оформляйте синтаксисом TypeScript — название метода начинается с подчеркивания. Например: _iconChangedValue.

```
1 // Хорошо
2 class BigSeparator extends Control<IBigSeparatorOptions> implements
  ICheckable {
3
4     private _iconChangedValue(value: boolean): void {
5         if (value) {
6             this._icon = 'icon-AccordionArrowUp ';
7         } else {
8             this._icon = 'icon-AccordionArrowDown ';
9         }
10    }
11 }
1 // Плохо
2 var protected = {
3     iconChangedValue: function (self, options) {
4         if (options.value) {
5             this._icon = 'icon-AccordionArrowUp ';
6         } else {
7             this._icon = 'icon-AccordionArrowDown ';
8         }
9     }
10 }
```

3. Хуки жизненного цикла — protected методы.

```
1 protected _beforeMount(newOptions: ICheckableOptions): void {
2     this._iconChangedValue(newOptions.value);
3 }
```

```

4
5 protected _beforeUpdate(newOptions: ICheckableOptions): void {
6     this._iconChangedValue(newOptions.value);
7 }

```

4. Обработчики событий – protected методы.

```

1 protected _clickHandler(): void {
2     this._notify('valueChanged', [!this._options.value])
3 }

```

Для аргумента типа события event используется

```

1 import {SyntheticEvent} from 'UICommon/Events';

```

5. defaultProps – [статическое поле](#), getOptionTypes – [статический метод](#).

```

1 static defaultProps: Partial<IOptions> = {
2     value: false
3 };
1 static getOptionTypes(): object {
2     return {
3         value: EntityDescriptor(Boolean)
4     };
5 }

```

6. У методов и переменных указывайте корректные модификаторы.

```

1 protected _icon: string;

1 protected _clickHandler(): void {
2     this._notify('valueChanged', [!this._options.value])
3 }

```

7. Расставляйте типы у полей, аргументов методов и возвращаемых значений методов.

```

1 protected _iconChangedValue(value: boolean): void {
2     if (value) {
3         this._icon = 'icon-AccordionArrowUp ';
4     } else {
5         this._icon = 'icon-AccordionArrowDown ';
6     }
7 }
1 protected _icon: string;

```

8. Тему и template оформляйте следующим образом (TemplateFunction импортируем из библиотеки UI/Base):

```

1 import {Control, IControlOptions, TemplateFunction} from 'UI/Base';
2 import 'css!Controls/toggle'

1 protected _template: TemplateFunction = checkBoxTemplate;

```

9. Контроль по умолчанию экспортирует себя.

```

1 // Хорошо
2 export default BigSeparator;

1 // Плохо

```

```
2 | export = BigSeparator;
```

Исключение: На время переходного этапа, если ваш контрол не является частью библиотеки (например, контрол SwitchableArea) и его могут грузить через require, надо писать export = для совместимости

10. Полное имя контрола в составе JavaScript библиотеки соответствует форме "Модульсбис/библиотека:контрол", например: Controls/list:View.

В этом примере:

- Controls — это имя [модуля СБИС](#). Он физически хранится [здесь](#), а также распространяется в составе [SBIS SDK](#).
- list — это имя JavaScript-библиотеки.
- View — краткое имя контрола в составе библиотеки.

Экспорт из библиотек

```
1 | // Хорошо
2 | export {default as BigSeparator} from
   | 'Controls/_toggle/BigSeparator';

1 | // Плохо
2 | import BigSeparator = require ('Controls/_toggle/BigSeparator');
3 |
4 | export {
5 |     BigSeparator
6 | }
```

Кастомные типы

Если в публичном методе используется какой-то сложный тип аргумента, который объявляется в вашем модуле, такие типы лучше тоже экспортировать. Они могут потребоваться при задании типов переменных в прикладном коде.

Пример: тип where – аргумент вызова фильтрации на источнике данных.

```
1 | type Where = IHashMap<string> | ((item: any, index: number) =>
   | boolean);
```

Набор утилит

Если модуль экспортирует утилиты: набор методов или констант, каждый метод/константу экспортируем отдельно.

В ts можно будет сделать удобный импорт нужных утилит по отдельности. А если вас импортят в js через require, то набор утилит придет в одном объекте и будет работать «как раньше».

```
1 | export const showType = (...);
2 |
3 | export function getMenuItems<T>(items: RecordSet<T> | T[]):
   | ChainAbstract<T>{
4 |     return factory(items).filter((item) =>{
5 |         return item.get('showType') !== this.showType.TOOLBAR;
6 |     })
7 | }
```

SyntheticEvent

```
1 | protected _onMouseDownHandler(event: SyntheticEvent<MouseEvent>): void
   | {
2 |     if (!this._options.readOnly) {
```

```
3     const box = this._children.area.getBoundingClientRect();
4     const ratio = Utils.getRatio(event.nativeEvent.pageX, box.left +
window.pageXOffset, box.width);
5     this._value = Utils.calcValue(this._options.minValue,
this._options.maxValue, ratio, this._options.precision);
6     this._setValue(this._value);
7     this._children.dragNDrop.startDragNDrop(this._children.point,
event);
8   }
9 }
```

См. также

- [Настройка и работа с TypeScript](#)